MiniPMDB Development Handbook

Architecture usage troubleshooting and continuing development

OchsSoft | 26 October 2026

This handbook explains the project from everyday use to the code and data that implement it. Use it to set up a development checkout, follow a request or document through the system, locate the right extension point, make a change, and verify the result.

Begin with the project overview and architecture. The later chapters provide exact local commands, user journeys, worked development examples, diagnostics, recovery steps, known gaps, and a pinned source reference. Technical terms are explained where they are used.

The documented source is OchsSoft/MiniPMDB at 834274e5bf7ab78bac36d6fd70ec317759201dcc. Source-defined behavior, observed evidence, planned capabilities, and remaining platform work are distinguished. The quick operations guide remains a separate short reference.

Contents

Understand the project

Understand the project....3
Architecture and data flow....4

Use and develop the project

Local development and setup....9
Using running and closing....11
Developing and testing changes....13

Diagnose improve and verify

Known gaps diagnosis and recovery....15
Source reference....16

Understand the project

What MiniPMDB is for

MiniPMDB is a local notebook for facts that software assistants and people need to share without quietly turning every suggestion into a rule. It stores project decisions, constraints, risks, bugs, and evidence. A human reviews candidates before they become active guidance. Its dashboard lets you see the review queue, project records, relationships between projects, and an audit report.

Think of a workshop with labelled folders. A project is a folder; a memory is a written claim; a source is the receipt that supports the claim. A touchpoint is a note saying that two folders depend on the same agreement. The context builder packs selected notes into a small briefing for the next piece of work. The audit checks whether those notes and agreements are internally consistent. Neither operation proves that a claim is true in the outside world.

This is a standalone memory-governance product. It is not an autonomous coding runner, shell executor, deployment service, transcript collector, or remote hosting platform. Connecting an assistant through MCP gives it a defined memory interface. It does not give MiniPMDB authority to execute the assistant's proposed work.

The three interfaces

The browser dashboard, CLI, and MCP server all use one foreground HTTP service. That service owns the MongoDB connection. The browser is the human review surface. The CLI is useful for repeatable, explicit commands. MCP is a line-oriented JSON-RPC interface used by an assistant application. MCP means Model Context Protocol: a common way for an assistant to discover named tools and call them.

There is no second private memory store inside the CLI or MCP process. If the service is stopped, those clients cannot read live records. Offline snapshot audit commands are a deliberate exception: they inspect an exported JSON file without contacting the service.

MongoDB is the canonical store, meaning it is the authoritative saved data. Context text is a derived view, like a printed agenda made from a filing cabinet. Do not edit a copied context pack and expect the database to change. The managed runtime uses persistent WiredTiger files under the application data directory. Despite the dependency's name, `mongodb-memory-server-core`, this configuration is not intended as a throwaway in-memory database.

Vocabulary you will meet in the code

A schema defines the allowed shape of a record. Normalization cleans and checks an input before storage. An index helps MongoDB locate records and, for unique indexes, prevent duplicate identities. A snapshot is an export of all five collections at schema version 2. Provenance means where a claim came from and how it was reviewed. Supersession means a newer record replaces an older one while preserving the older record as history.

A candidate has status `draft` or `unreviewed`. Active statuses include `current`, `reviewed`, `open`, and `resolved`, subject to review metadata. A warning record may be draft, unreviewed, stale, or superseded. Archived and rejected records are excluded from ordinary context selection. Confidence is a separate field: `high` is an assertion of confidence, not a substitute for evidence or review.

A conflict link says that two claims disagree. Resolving it requires a reviewed active resolution record, rather than hiding the old disagreement. A touchpoint describes a cross-project API, shared library,

data contract, workflow, deployment relationship, or other connection. Its project keys and referenced memories must belong together.

Current implementation and evidence boundary

The source implements a loopback dashboard/API, persistent managed or external MongoDB storage, project registration, candidate capture, evidence attachment, human review, supersession, touchpoints, three context profiles, and structural governance audits. Node's built-in test runner covers service, API, MCP, runtime, context, and audit behavior. The application dependencies include the MongoDB driver and the managed MongoDB lifecycle library; this is not a dependency-free program.

This handbook is a source investigation at the pinned revision in the final chapter. It does not claim that a new database was launched, an assistant was connected, or the MongoDB binary was qualified on this NixOS machine while the handbook was written. The documentation's build and formatting checks are separate from those runtime acceptance tests. The existing short operations guide remains the quick reference once you know the system.

Trust and privacy

The server binds to `127.0.0.1` and has no user login or per-user access-control system in the inspected API. A process that can reach the loopback API can call human-powerful routes, including review and demo reset. Restricting MCP to draft writes protects that MCP interface; it does not turn the underlying local HTTP API into an authenticated multi-user service.

Use a trusted local account, avoid port forwarding or public reverse proxies, and store only appropriate project facts. A source reference should point to evidence, not embed credentials or private transcripts. Export files contain the stored material and local project paths; treat them as private backups. This trust model is appropriate to explain clearly before expanding the product, rather than assuming loopback means every possible local caller is harmless.

Architecture and data flow

Components and ownership

`src/api.js` starts `RuntimeManager`, constructs `MiniPMDBService`, routes requests, and serves the dashboard. `src/runtime.js` owns MongoDB lifecycle. `src/database.js` initializes collections, validators, indexes, snapshots, and connection cleanup. `src/service.js` owns record operations and review policy. `src/schema.js` normalizes data. `src/context.js` selects and formats briefings. `src/audit.js` checks integrity. `src/api-client.js` is shared by the CLI and MCP client surface. `public/app.js` renders dashboard state and submits human actions.

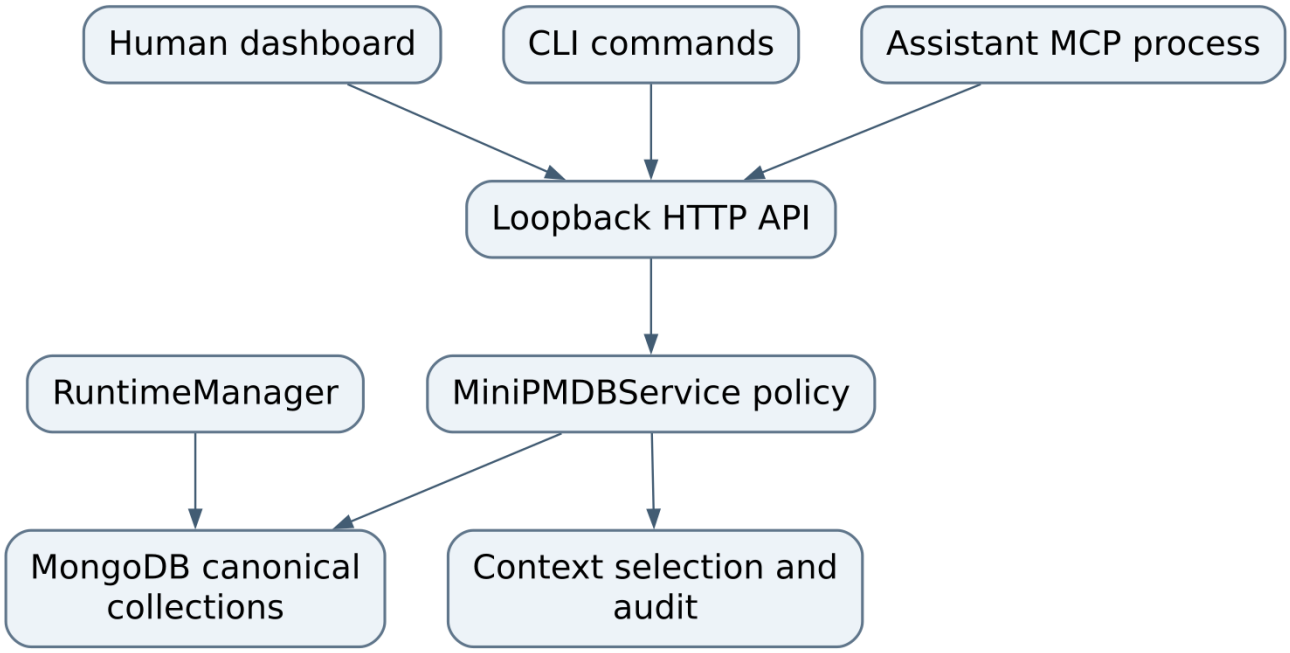


Figure 1 Components and ownership

The service is the correct place for a rule shared by several interfaces. A browser-only restriction can be bypassed by the CLI. An MCP-only restriction does not protect the dashboard/API. The database layer should handle persistence mechanics, while service methods express operations such as “attach evidence to an unreviewed candidate.”

The five collections

Collection	Identity and important fields	Meaning
projects	`id`, unique `key`, `name`, `repo_path`, canonical `repo_root`, status, tags, timestamps	Registered project and repository identity
memories	`id`, `project_key`, kind, status, confidence, title, body, tags, `source_ids`, critical flag, metadata	A claim and its governance state
sources	`id`, `project_key`, type, label, `ref`, creation time	Evidence reference owned by a project
links	`id`, relationship, `from`, `to`, reason, status, `resolution_memory_id`, timestamps	Supersession, conflict, or support relationship

Collection	Identity and important fields	Meaning
touchpoints	`id`, name, project keys, kind, status, summary, memory IDs, tags, verification date	Cross-project agreement and its evidence

Memory titles are bounded to 240 characters and bodies to 8,000 in normalization. Touchpoint summaries are bounded to 2,000. Metadata is an object, not an unrestricted substitute for a new schema. Snapshot normalization requires version 2, all five arrays, and unique IDs within each collection. A structurally valid snapshot may still fail governance audit because a referenced project or source is absent.

MongoDB initializes unique IDs and project keys, a unique nonempty repository-root index, and query indexes for project/status/date and relationship endpoints. Empty repository roots do not collide with one another. The initializer removes an older unique `repo_root_1` index before adding the current partial index. This is startup schema/index management, not a complete versioned migration framework.

Repository identity

Registration resolves the provided path and attempts `realpath`; on Windows the resolved root is lowercased. MCP resolves its current working directory against an exact registered `repo_root`. Starting MCP from an arbitrary nested directory is therefore not equivalent to registering a repository's root. A moved checkout or a different worktree path may need an intentional registration update. Do not register a near-looking project key merely to make a failed lookup disappear.

`registerProject` upserts by project key and preserves the original creation timestamp. `remember` requires a registered project. It generates a memory ID when none is supplied, stamps creation/update times, and forces candidate status when `reviewFirst` is true. `attachSource` inserts a source then adds its ID to the memory. The project-scoped MCP path additionally checks ownership and candidate status.

Candidate to reviewed guidance

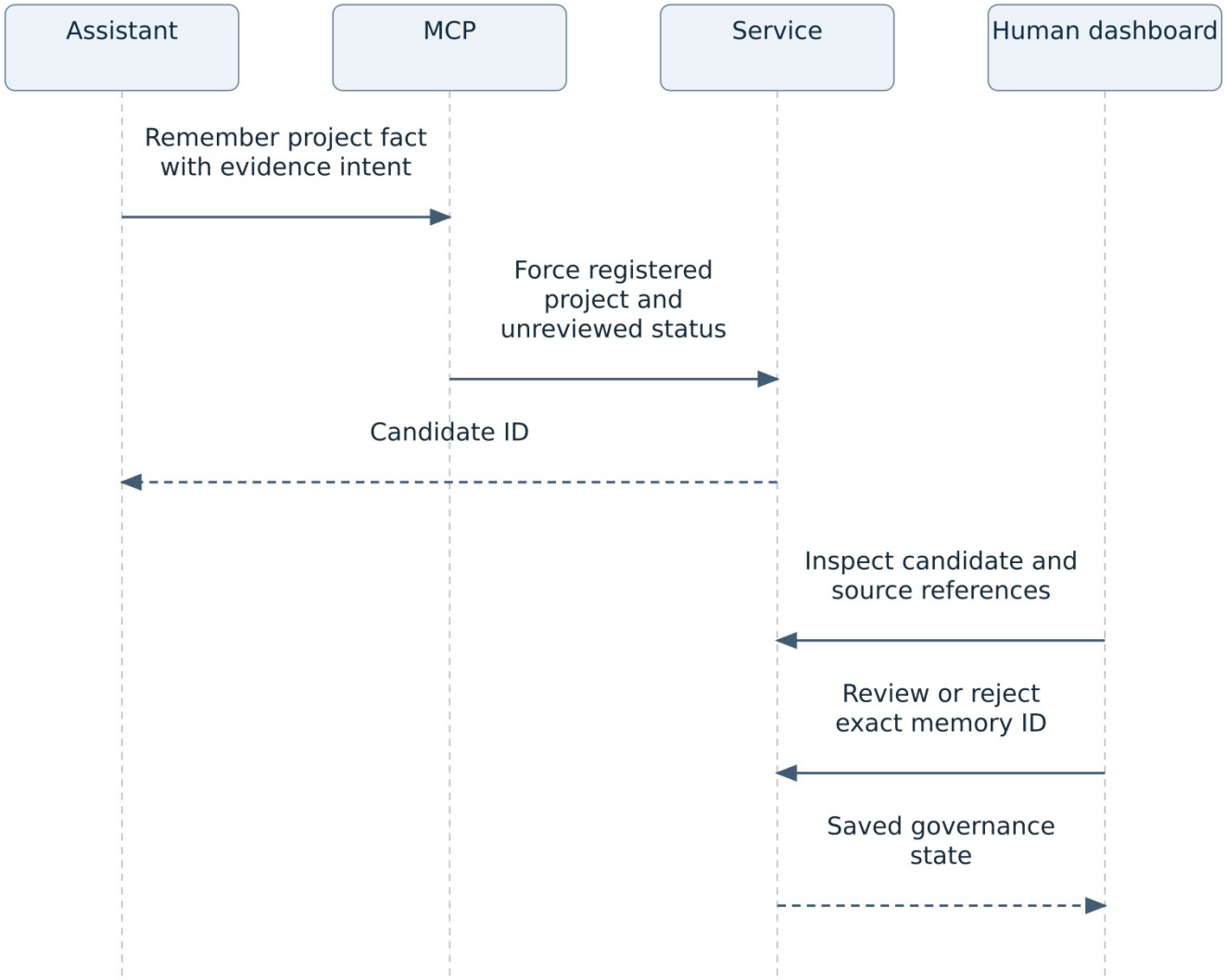


Figure 2 Candidate to reviewed guidance

Human review records reviewer and review note in metadata. Allowed review targets are reviewed, current, rejected, or archived. A caller-supplied reviewer string is a label rather than cryptographic proof of who clicked. Superseding checks that both records exist, updates the old record's status/metadata, and writes a link from replacement to old record. These are multiple writes; the inspected implementation does not wrap them in a transaction.

Context selection in detail

`buildContextPack` first normalizes the snapshot and requires the requested project. It excludes archived/rejected memories. Own-project memories are ranked by status, task-word matches, critical flag, and conflict warnings. Task words use a simple lowercase token matcher, not semantic embeddings. Ties use newest `updated_at`, then lexicographic ID, making repeated inputs deterministic.

Cross-project memories are reached through a touchpoint's explicit `memory_ids`. They are not retrieved by unrestricted search across every other project. Broken touchpoints and critical warnings receive required-first selection. Own active records are capped at 12 before text trimming. Profile defaults are:

`drift_guard` 12,000 characters, `balanced` 7,000, and `compact` 3,500; their body excerpts, warning limits, and touchpoint/cross-project limits also differ.

The formatter writes active truth, cross-project context, warnings/history, and touchpoints. It trims optional entries when over budget. If required content still exceeds the budget, it finally slices the string and adds `[TRUNCATED]`. This last fallback is a real limitation: selected-ID telemetry can still report a critical warning as selected even if the text was cut before its full content. A small pack is not proof that every governing fact was delivered. Use the underlying records and audit output before consequential work.

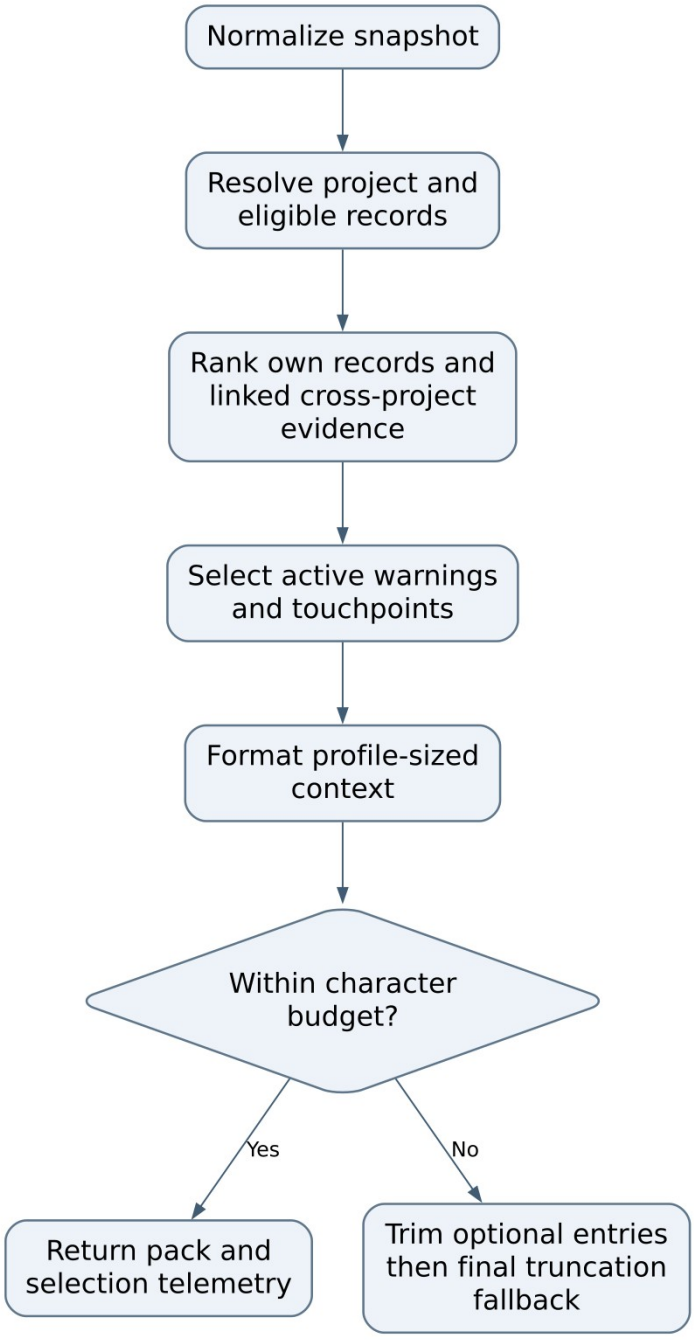


Figure 3 part 1 of 2 Context selection in detail

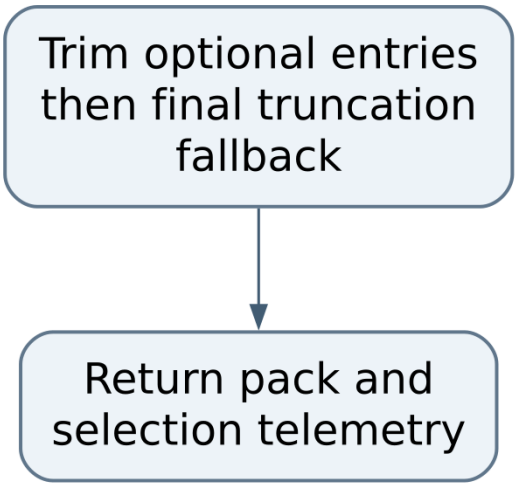


Figure 3 part 2 of 2 Context selection in detail

Audit behavior

`auditSnapshot` checks the raw references before it builds context for all three profiles. Errors include missing projects, broken source/link references, cross-project source ownership, active records marked unreviewed, high-confidence active claims without sources, unresolved conflicts, invalid resolution records, and incomplete active/broken touchpoints. Candidate review and stale/broken status can produce warnings. Strict mode fails on either errors or warnings; ordinary mode fails only on errors. An awaiting-review warning can be expected work, but it must not be misreported as a strict pass.

Local development and setup

Start with a safe checkout

Use a topic branch based on the repository's merged `master`, keep the working tree's unrelated changes, and read `package.json` before installing. Node 20 or newer is the documented baseline; use the actual dependency lockfile and the project's supported current Node release when qualifying a build. Do not install a second package manager just for the handbook.

```
git status --short
git branch --show-current
node --version
npm ci
npm run check
npm test
npm run smoke
```

These commands are instructions for local qualification, not a claim that application tests were executed during documentation authoring. `check` runs the repository checker; `test` uses the built-in Node runner serially; `smoke` audits the resolved synthetic fixture and checks snapshot context. Read the scripts before expanding the gate. Keep routine hosted validation disabled under the repository owner's local-first policy.

Decide which MongoDB mode you intend

The ordinary source default is managed mode. `RuntimeManager.start` creates the application home, `data/`, and binary-download cache. It requests MongoDB 8.2.6, MD5 verification, loopback binding, persistent WiredTiger storage, and a managed child process. The first start downloads a platform-specific server binary. Network success does not prove binary compatibility on NixOS.

On NixOS, a downloaded generic Linux executable can fail because its dynamic loader or shared libraries are absent from the expected filesystem paths. This repository does not provide a checked-in Nix flake establishing managed-mode compatibility. Treat that path as unverified until a real start, persistence restart, and shutdown have been demonstrated on the target environment. Do not respond to a loader failure by deleting the database.

The external mode avoids having the Node process own a downloaded MongoDB binary. The supplied `compose.yaml` describes only a MongoDB service with image `mongo:8.2.6-noble`, a named data volume, and host mapping `127.0.0.1:27017:27017`. Use Podman and an available Compose provider; do not use Docker. Check the port before starting because another database may already own it.

```
podman compose -f compose.yaml up -d mongodb
MINIPMDB_MONGODB_URI='mongodb://127.0.0.1:27017' npm start
```

The example explicitly matches the Compose host port. The persisted external-mode default URI is port 27019; copying that default while starting the supplied port-27017 service will fail. Do not change an unrelated existing MongoDB service to compensate. The container's healthcheck pings MongoDB; the application still needs its own readiness check and database name verification.

Configuration and precedence

MINIPMDB_HOME

Source default or behavior. OS application data directory

Operational consequence. Contains config and managed state, never a repository backup substitute

XDG_DATA_HOME

Source default or behavior. Linux fallback `~/.local/share`

Operational consequence. Default Linux application subdirectory is `minipmdb`

MINIPMDB_API_URL

Source default or behavior. Client loopback URL, default port 8797

Operational consequence. CLI/MCP must address the same service

MINIPMDB_MONGODB_URI

Source default or behavior. Unset

Operational consequence. When set, forces external mode regardless of saved mode

MINIPMDB_DB_NAME

Source default or behavior. `minipmdb` before saved-config merge

Operational consequence. Verify database identity before creating records

MINIPMDB_MONGODB_VERSION

Source default or behavior. 8.2.6 before saved-config merge

Operational consequence. Managed binary version; changing it is an upgrade decision

MINIPMDB_MONGODB_DOWNLOAD_DIR

Source default or behavior. Managed `binaries/` directory

Operational consequence. Binary cache location, distinct from canonical `data/`

MINIPMDB_MCP_MODE

Source default or behavior. `project-draft`

Operational consequence. Candidate-only writes by resolved repository scope

`config.json` contains mode, database name, managed version, and a credential-free external URI. Saved values are merged over defaults, while the explicit MongoDB URI environment variable takes precedence at runtime. Saving runtime configuration returns `restart_required: true`; it does not hot-switch a live connection. Credentialed URIs are rejected by the persisted-config validator and must come from the environment. Do not paste credential values into a command that will be saved in shared shell history or documentation.

Confirm readiness before writing

Start the service in a terminal you control. Open the loopback dashboard manually if desired, inspect runtime status, and verify mode, endpoint, database, and MongoDB version. A reachable HTML page alone is not the full readiness proof. Read `/api/health` and `/api/runtime` or use the CLI:

```
node src/cli.js runtime
node src/cli.js list --project sample-workshop
```

The list command assumes that synthetic project has already been registered. An empty result is different from a refused connection or unknown registration. The dashboard's project selector is driven by stored projects; creating a browser bookmark does not register one.

For an isolated demonstration use a separate `MINIPMDB_HOME`, database name, and intended port. Demo reset is destructive to the selected database. Never run a demo reset against an existing personal database merely to populate the interface. Synthetic fixtures are available for offline checks without modifying a live service.

Using running and closing

Register one repository

Run the CLI against the intended loopback API, supplying a deliberate stable key and the actual repository root. This example uses a synthetic folder that you create for the exercise:

```
mkdir -p /tmp/mini-workshop-demo
node src/cli.js project add --key sample-workshop \
  --name 'Sample Workshop' --repo /tmp/mini-workshop-demo
```

Read the returned project key and canonical root. If the path is wrong, correct registration deliberately before starting MCP. A project key is an identity that other records reference; renaming a display label is different from migrating every reference to a new key.

Record support and review a fact

Create a narrowly phrased candidate describing a decision and its reason. The CLI forces review-first capture. Do not use one huge memory to hold a whole log or every open task.

```
node src/cli.js remember --project sample-workshop \
  --title 'Use a local-only development listener' \
  --body 'The development API binds to loopback so this demo is not exposed on the LAN.' \
  --kind constraint --confidence medium
```

Copy the returned memory ID. Attach a real synthetic evidence reference using `source attach MEMORY_ID --label LABEL --ref REF`. Then inspect the claim and evidence in the review queue. Choose approve only when the actual source supports the wording and scope. Reject unsupported candidates; review is not a cosmetic step to make audit green. The dashboard writes reviewer `dashboard-human` and a standard note; the CLI permits an explicit reviewer/note for a more useful audit trail.

After review, request context for the same project and a concrete task. Read warning/history sections as well as active guidance. A superseded decision can still explain why an older source file differs from the current intended policy.

```
node src/cli.js context --project sample-workshop \
  --task 'change the development listener' --profile balanced
node src/cli.js audit --project sample-workshop --strict
```

Change a decision without erasing history

First create and review the replacement record with the new reason and supporting evidence. Then use `supersede OLD_ID --with REPLACEMENT_ID --reason TEXT`. Check the old record, replacement, and supersession link afterward. A successful single request is not permission to delete the old memory. If interrupted, inspect both records and links before retrying because the operation is not transactional and a repeated call can create another link.

A touchpoint is appropriate when two registered projects share an actual contract. Use `touchpoint upsert` with both project keys and a memory from each side. The service checks that each memory exists and belongs to a participant. For an active relationship the audit additionally expects a nonexcluded memory for every participant. Use `planned` for a proposed connection and change its state only when the contract has been checked.

Connect an assistant safely

Start the HTTP service first. Configure the assistant's MCP client to launch `node src/mcp.js` from the exact registered repository root, with the intended `MINIPMDB_API_URL`. Keep stdout reserved for protocol messages; diagnostic text belongs on stderr. Do not launch it from an unrelated directory and then grant a broader mode to fix a scope error.

Default `project-draft` exposes read tools and project-scoped candidate/evidence writes. `read-only` removes write tools. The accepted alias `draft-write` follows the draft-write policy, not unrestricted human review authority. Tool descriptions and annotations are informative; enforced checks in the server/service remain essential. The MCP interface does not offer human review, arbitrary project registration, or unrestricted supersession.

Ask the assistant to capture a synthetic fact, confirm the returned ID, then independently see that it is still unreviewed. Verify that attempts to write another project, attach evidence to an already reviewed record, and invoke unavailable human operations fail in a test fixture. A tool's existence in a client list is not evidence that a memory was actually saved.

Export and close

Before a substantial change, write an export to a private backup directory:

```
node src/cli.js export --out /tmp/minipmdb-workshop-backup.json
node src/cli.js audit-snapshot \
  --snapshot /tmp/minipmdb-workshop-backup.json --strict
```

Use a durable private location instead of `/tmp` for a real backup. The temporary path here avoids suggesting a universal machine-specific backup layout. Export includes all collections, not only the currently selected dashboard project. Snapshot reads are parallel across collections, so a busy database can change during export; quiesce writes for an important recovery point.

Stop the MCP client/assistant connection, finish outstanding writes, and stop the foreground API with its normal terminal interrupt. `RuntimeManager.stop` closes the driver and stops the managed child with cleanup disabled so persistent files remain. For an external Podman service, stop only the service you started after other clients have disconnected. Do not remove volumes. On the next start verify the same database identity and a known record ID before resuming work.

Developing and testing changes

Read the full request path

For a dashboard feature trace DOM controls in `public/index.html` and `public/app.js`, then its HTTP route in `src/api.js`, service method, normalization, and database write. For CLI/MCP changes also trace `src/cli.js`, `src/mcp.js`, and `src/api-client.js`. A useful first search is the route string or method name. Do not add a second storage helper because a frontend file seems convenient.

Use small focused tests with synthetic projects. The repository separates runtime, API, service, context, MCP, and audit tests. The service tests are where write-policy behavior belongs; context tests should assert both selected metadata and rendered text; API tests verify status/shape and lifecycle cleanup. A test that merely repeats a constant from the implementation does not establish a user-visible outcome.

Worked change make truncation honest

Problem: `buildContextPack` can preserve a critical warning in selection arrays but cut the final formatted string. `auditContext` checks selected omission arrays, not whether the full required text reached the final pack. This repair is proposed work, not present behavior.

First build a fixture with several long critical warnings, a broken touchpoint, and a deliberately small `maxChars`. Assert that either all required safety content appears or the result explicitly reports required overflow and incomplete governing coverage. Also assert the exact rendered pack budget and that a caller cannot mistake incomplete coverage for success. Include a warning with its important condition at the end of the body so a superficial ID-only check cannot pass.

Change `src/context.js` to track the items actually emitted after trimming, record required overflow, and avoid a silent arbitrary slice through a safety record. Decide whether overflow returns a bounded summary plus retrieval instructions or an explicit failure; document that contract before updating callers. Update `src/audit.js` to fail when required content cannot be delivered, then `public/app.js` to display the warning. Cover all three profiles and a normal no-overflow pack so the fix does not turn every request into an error. The acceptance evidence is actual output text and telemetry that agree, not just a green length assertion.

Worked change strengthen supersession integrity

Problem: `supersede` checks existence but does not reject self-replacement, prevent cycles, require an already reviewed replacement, or provide a single atomic relationship update. Those constraints are useful future governance work.

Write service tests for old equals new, replacement from an inappropriate project, unreviewed replacement, an existing chain/cycle, duplicate retry, and injected failure after updating the old record. Decide explicitly whether cross-project supersession is allowed; do not infer an identity migration from a similar title. Define idempotence: submitting the same completed request twice should return the same relationship rather than create duplicates.

Implement shared checks in `src/service.js`; keep schema validation in `src/schema.js` and storage operations in `src/database.js`. If using MongoDB transactions, establish a compatible replica-set deployment and test it; standalone managed MongoDB cannot simply be assumed to support transaction semantics. A recovery-oriented alternative may use a durable operation record and reconciliation, but that changes the data model and needs its own migration. Acceptance includes a restart/failure test demonstrating that the old memory cannot be left misleadingly superseded without a durable replacement relationship.

Worked change add a simple memory field

Suppose you want an optional review-due date. Define its meaning first: does it warn when overdue or automatically make guidance stale? Prefer an explicit warning rather than silently changing governance. Extend normalization in `src/schema.js`, the Mongo validator in `src/database.js`, service creation/update paths, snapshot import/export compatibility, dashboard rendering, and CLI/MCP schema only where those callers should set it. Existing records must remain readable when the field is absent.

Test invalid dates, absent dates, boundary times, and exports from the old shape. Use a fixed clock or supplied timestamp rather than tests that fail depending on today's date. Add an audit issue and UI explanation if overdue dates matter. Do not put a behaviorally important field only inside arbitrary metadata and rely on developers to remember its spelling.

Validation sequence

Run the narrow affected tests while iterating, then `npm run check`, `npm test`, `npm run smoke`, and `npm run sanitize` on the complete documentation/source tree. For API/runtime changes, add an isolated real MongoDB round trip: create, review, export, stop, restart, retrieve the same ID, and verify shutdown leaves no owned child running. Separate failures caused by unsupported downloaded binaries from failures in the JavaScript logic.

A docs-only change needs link, source-reference, diagram, Word, and PDF checks; it does not warrant a destructive demo reset. A future runtime PR should record exact Node/Mongo versions, platform, commands, counts, and remaining skips. Commit intentionally on a topic branch, push only after local proof, open a ready independent PR against `master`, and leave merge/release to the human. Hosted CI being disabled is not a passing check.

Known gaps diagnosis and recovery

Diagnose from the symptom

If the CLI reports a connection refusal, inspect the foreground API terminal and intended API URL before changing the database. If the dashboard loads but runtime is not ready, inspect mode, database, managed binary preparation, and Mongo connection errors. If records appear missing after a restart, compare `MINIPMDB_HOME`, saved configuration, URI override, and database name with the previous run. A different empty database is not proof that the original files were lost.

If managed start fails on NixOS, preserve `data/`, record the exact loader/library message, and qualify either a packaged executable configuration or the external Podman route. A failed binary download belongs to network/cache diagnosis; an executable that downloads then cannot start belongs to platform diagnosis. Repeatedly clearing data conflates both and destroys the strongest recovery evidence.

If MCP cannot resolve a project, compare its launch working directory with the registered canonical root. If a write tool is absent, inspect `MINIPMDB_MCP_MODE` and the client tool refresh. If a write succeeds but context does not show it as active, check candidate status, review metadata, ranking, profile limits, and truncation. Do not promote a record to high confidence solely to improve its ranking.

Audit failures are actionable records

`broken_source_reference` requires locating the missing source or correcting the claim's reference, not deleting the audit message. `cross_project_source_reference` means evidence ownership is wrong. `active_unreviewed` signals inconsistent governance state. `unresolved_conflict` requires an actual reviewed resolution. `touchpoint_missing_project_memory` means one side of a shared agreement has no qualifying record. Strict-mode candidate warnings usually mean review work remains, not database corruption.

The audit is structural. It does not fetch every source URL, verify a human identity, prove every stored claim, or execute the projects described. Its touchpoint check counts nonexcluded records; it is not an independent semantic review that every participant agreed. A passing report must be interpreted within those limits.

Recovery and backup limits

An export is useful but not automatically a transactionally consistent backup. `snapshot()` reads collections in parallel, while writes may proceed. `replaceSnapshot()` deletes and inserts collections sequentially and is destructive; interruption can leave a partial state. The ordinary CLI exposes export and offline audit, not a safe general-purpose atomic import wizard. The demo reset route is not a restore procedure.

For a real restore, stop writers, preserve the original database/volume and export, validate the candidate snapshot offline, and restore into a separate named database first. Compare collection counts, important IDs, sources, review metadata, links, and all-profile audits. Start an isolated API against that copy and verify representative records before any deliberate switch. Preserve the original until the human accepts the result. A copy of live WiredTiger files without proper database backup handling is not a guaranteed recoverable image.

Prioritized development backlog

The first priority is truthful context completeness: required warnings may survive selection but be cut by the final text slice. Fix `src/context.js`, `src/audit.js`, and dashboard warnings with overflow tests described in the development chapter. The acceptance criterion is that required text delivery and omission telemetry agree for tiny budgets and many critical records.

The second priority is multi-write integrity. Evidence attachment and supersession can stop between writes, while snapshot replacement can stop between collections. Design idempotent operations, validation, and recoverable failure handling. Do not advertise an atomic backup/import capability until a killed-process/fault-injection test proves it. Preserve raw broken states as synthetic fixtures for regression tests.

The third priority is the local trust boundary. The API's loopback binding is useful but has no authentication, human identity verification, or distinct authorization for review/demo operations. Before any shared-machine, forwarded-port, or remote access design, add an explicit threat model and authenticated authority checks. Test unauthorized review/reset, origin/CSRF behavior, credential handling, and restricted read/write scopes. Do not broaden the listener as a quick convenience change.

The fourth priority is reproducible NixOS qualification. Add an approved development environment and a tested database route, then demonstrate persistence, restart, cleanup, and offline recovery. Record managed and external modes separately. A generic Linux build badge cannot settle NixOS loader compatibility.

The fifth priority is scale and migrations. Several endpoints materialize the full snapshot, and context/audit use in-process scans. Add bounded pagination and explicit full-record retrieval without confusing a bounded page with an exhaustive project inventory. Introduce versioned migrations and compatible snapshot upgrades before changing stored contracts. Measure with synthetic large datasets; never solve a slow context request by silently dropping the warnings it was meant to preserve.

Before you stop a difficult session

Write the reproducible issue in the normal issue/PR system with the exact source revision, relevant record IDs, redacted error, command, expected result, observed result, and preservation location. Keep secrets and raw private records out of public reports. Leave the database stopped or clearly identify the owned running service. This handbook supplies the map; durable project claims should still pass through the product's evidence and review process rather than becoming informal notes scattered around the repository.

Source reference

The handbook describes the committed implementation at the following baseline. These links remain tied to that revision, so a future change can be compared with the behavior described here.

Repository `OchsSoft/MiniPMDB`. Baseline `834274e5bf7ab78bac36d6fd70ec317759201dcc`.

Source	What to inspect
src/api.js	Loopback server, routes, static files, startup and close.

Source	What to inspect
src/runtime.js	Managed/external storage lifecycle and persistent WiredTiger path.
src/runtime-config.js	Application home, config validation and credential rule.
src/database.js	Collections, indexes, snapshot and nontransactional replacement.
src/schema.js	Normalization and snapshot version 2.
src/constants.js	Kinds, statuses, context profiles and pinned MongoDB.
src/service.js	Project identity, candidate policy, review, sources and supersession.
src/context.js	Ranking, required-first selection, trimming and final slice.
src/audit.js	Integrity issue codes and three-profile audit.
src/cli.js	Human commands, export and offline audit.
src/mcp.js	JSON-RPC, tool modes and repository scope.
src/api-client.js	Shared HTTP client and default endpoint.
public/app.js	Dashboard state, review queue, runtime controls and context display.
public/index.html	Human controls and labels.
compose.yaml	Optional external MongoDB service; use Podman.
package.json	Actual check/test/smoke/sanitize commands and dependencies.
test/context.test.js	Context profile and warning regression tests.
test/service.test.js	Service behavior tests.
test/mcp.test.js	MCP policy and protocol tests.
test/api.test.js	API contract tests.
test/runtime.test.js	Storage lifecycle tests.
test/audit.test.js	Audit safety checks.

Source	What to inspect
scripts/sanitize-check.js	Public-source vocabulary, file-size and credential-like checks.

Use repository search to find a named function before editing. Source line numbers move as code changes; a symbol and pinned commit are more durable together than an unqualified line number.

How to keep this handbook current

When a route, schema, mode default, storage path, or review rule changes, update the affected chapter and its diagram in the same PR. Keep the source baseline explicit; compare the new source with this edition before relying on a formerly known limitation. Regenerate Word and PDF from the reviewed chapter text and inspect every page. A planned repair remains a gap until code and its real acceptance evidence establish otherwise.

The reproducible commands in this book are source-defined instructions. Documentation creation does not certify live persistence, browser use, assistant connection, or managed MongoDB qualification. Record those as separate local validation results when you actually perform them.